



MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)
SEMESTER 1, 2025/2026

WEEK 8: BLUETOOTH DATA INTERFACING

SECTION 1

GROUP 20

LECTURER: DR. WAHJU SEDIONO

DATE OF EXPERIMENT: Monday, 1st December 2025

DATE OF SUBMISSION: Monday, 8th December 2025

PREPARED BY:

NAME	MATRIC NUMBER
ALI RIDHA BIN MUHAMMAD AMINUDIN	2316137
SITI NORATHIRAH BINTI RAZALLY	2319788
NUR ALYA SAKINAH BINTI MOHD MOKHTAR	2319444

ABSTRACT

This project explores a simple wireless monitoring and control system using Bluetooth communication between a microcontroller and a paired device such as a smartphone. The system continuously reads temperature data from a DHT22 sensor and transmits the real-time readings through a Bluetooth serial link. At the same time, the microcontroller is programmed to receive control commands, specifically “LED ON” and “LED OFF” which allow the user to remotely toggle an LED.

By combining sensor data transmission with command-based control, the project demonstrates the essential concepts of Bluetooth interfacing, data acquisition, and remote actuation. This setup shows how environmental monitoring and simple device control can be integrated into a single, user-friendly wireless system. Overall, the experiment serves as a foundational step toward more advanced IoT-based automation and smart monitoring applications.

TABLE OF CONTENTS

ABSTRACT	2
TABLE OF CONTENTS	3
1.0 INTRODUCTION	4
2.0 MATERIALS AND EQUIPMENTS	5
3.0 EXPERIMENTAL SETUP	6
4.0 METHODOLOGY	8
4.1 Circuit Assembly	8
4.2 Programming Logic	9
4.3 Code Used	12
4.4 Control Algorithm	15
5.0 DATA COLLECTION	16
6.0 DATA ANALYSIS	17
7.0 RESULT	18
8.0 DISCUSSION	19
9.0 CONCLUSION	20
10.0 RECOMMENDATIONS	21
11.0 REFERENCES	22
12.0 APPENDICES	23
Certificate of Originality and Authenticity	27

1.0 INTRODUCTION

Bluetooth technology has become an essential component in modern electronic systems because of its reliability, simplicity, and effectiveness in supporting short-range wireless communication. In this experiment, Bluetooth is used to design a compact system that combines temperature monitoring with remote LED control. Through a paired device, such as a smartphone or laptop, the user receives live temperature readings from a DHT11 sensor while also being able to send commands like “LED ON” and “LED OFF” to control the microcontroller’s output. This setup illustrates how wireless communication can be applied not only to collect environmental data but also to influence a device’s behavior in real time.

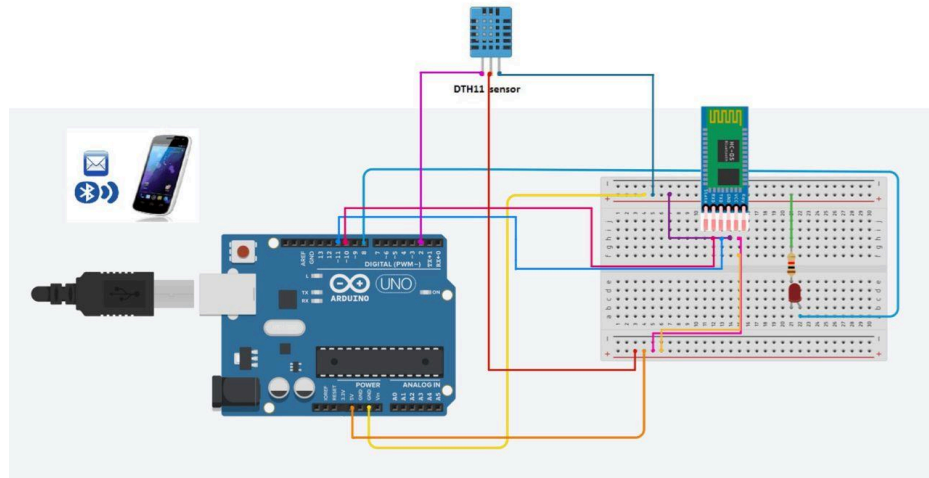
The experiment is supported by core concepts in embedded system design, including serial communication, digital sensing, and output control. The Bluetooth module functions by emulating standard UART communication, allowing the microcontroller and external device to exchange data in a simple text-based format. Meanwhile, the DHT11 sensor provides accurate digital temperature readings that can be processed and transmitted with minimal complexity. By integrating these components, the experiment demonstrates how sensing, data transmission, and command-based actuation can operate seamlessly within a single wireless platform.

The purpose of this experiment is to establish a fully functional Bluetooth link capable of delivering continuous temperature data while also enabling the microcontroller to receive and interpret user-sent control commands. The expected outcome is that the system will transmit temperature readings reliably and that the LED will respond immediately and correctly to any incoming command. Overall, the experiment is anticipated to produce a stable and responsive wireless communication setup that effectively showcases the integration of monitoring and control functions.

2.0 MATERIALS AND EQUIPMENTS

1. Microcontroller Board - Arduino UNO
2. Bluetooth Module - HC-05
3. Temperature Sensor - DHT11
4. LED Indicator
5. Resistors
6. Breadboard
7. Jumper Wires
8. Power Supply
9. Smartphone with Bluetooth
10. Bluetooth Serial Communication App
11. USB Cable for Programming

3.0 EXPERIMENTAL SETUP



Hardware Configuration

1. Arduino UNO
 - The Arduino UNO acted as the main processing unit, reading temperature data from the DHT11 sensor and handling Bluetooth communication via the HC-05 module.
2. Temperature Sensor (DHT11)
 - Connected the data pin of the DHT11 to digital pin 2 on the Arduino.
 - The VCC and GND pins of the sensor were connected to the Arduino's 5V and GND, respectively.
 - The sensor measured ambient temperature and humidity in the test environment.
3. Bluetooth Module (HC-05)
 - Connected the HC-05 module to digital pins 10 (RX) and 11 (TX) on the Arduino using SoftwareSerial.
 - The module enabled wireless communication with a paired smartphone or PC to transmit temperature data and receive control commands "LED ON" or "LED OFF".
 - VCC and GND of the HC-05 were connected to the Arduino's 5V and GND pins.
4. LED for Control Simulation
 - The LED was connected to digital pin 8 with a resistor.
 - Controlled remotely via Bluetooth commands from a paired device.

5. Power Supply

- The Arduino UNO was powered via USB from a computer or an external 5V supply.
- Both the DHT11 sensor and the HC-05 module received power directly from the Arduino.

6. Breadboard and Jumper Wires

- Components were connected on a breadboard using jumper wires to ensure stable and organized circuitry.

Software Setup

1. Arduino Programming

- The Arduino IDE was used to write a sketch that initialized the DHT11 sensor and the HC-05 Bluetooth module.
- Temperature readings were obtained every 2 seconds and sent over Bluetooth.
- The code was also designed to receive simple control commands from the paired device to simulate environmental control, such as toggling an LED to represent a fan.

2. Bluetooth Communication

- The HC-05 module was paired with a smartphone or PC.
- A serial terminal app or Python script was used to view temperature readings in real-time and send control commands back to the Arduino.

Physical Arrangement

- The DHT11 sensor was placed in the environment to measure ambient temperature accurately.
- The Arduino UNO and HC-05 module were placed on the breadboard near the sensor to maintain compact and organized wiring.
- The paired device was within Bluetooth range (~10 meters) to allow uninterrupted wireless communication.

4.0 METHODOLOGY

4.1 Circuit Assembly

1. Connection for temperature sensor DHT11 to Arduino Uno
 - Pin + $\rightarrow$ VCC
 - Pin - $\rightarrow$ GND
 - Pin S $\rightarrow$ D2
2. Connection for Bluetooth module HC-05 to Arduino Uno
 - Pin VCC $\rightarrow$ VCC
 - Pin GND $\rightarrow$ GND
 - Pin TXD $\rightarrow$ D10
 - Pin RXD $\rightarrow$ D11
3. Connection LED to Arduino Uno
 - Anode (+) $\rightarrow$ D8
 - Cathode (-) $\rightarrow$ GND

4.2 Programming Logic

1. Library Import & Definitions

```
#include "DHT.h"

#include <SoftwareSerial.h>

#define DHTPIN 2           // DHT data pin

#define DHTTYPE DHT11

#define LED_PIN 8          // <-- wire LED to pin 8
```

- Bring in required libraries (DHT sensor + Bluetooth serial communication)
- Define the pins used for DHT and LED

2. Object Initialization

```
DHT dht(DHTPIN, DHTTYPE);

SoftwareSerial bluetooth(11, 10); // HC-05 (RXD, TXD)
```

- Create DHT sensor object
- Create Bluetooth communication on digital pins 11 & 10

3. Setup Configuration (Runs Once)

```
void setup() {

  Serial.begin(9600);

  bluetooth.begin(9600);

  dht.begin();

  pinMode(LED_PIN, OUTPUT);           // <-- LED declared as output

  digitalWrite(LED_PIN, LOW);        // LED initially OFF

  Serial.println("DHT11 + Bluetooth Ready");

  bluetooth.println("Bluetooth LED + DHT11 Ready"); }
```

4. Main Loop (Runs Continuously)

```
void loop() {
```

4A. Bluetooth Command Handling Logic

```
    if (bluetooth.available()) {  
  
        String cmd = bluetooth.readStringUntil('\n');  
  
        cmd.trim();  
  
        if (cmd == "LED ON") {  
  
            digitalWrite(LED_PIN, HIGH);  
  
            bluetooth.println("LED Turned ON");  
  
            Serial.println("LED ON");  
  
        }  
  
        else if (cmd == "LED OFF") {  
  
            digitalWrite(LED_PIN, LOW);  
  
            bluetooth.println("LED Turned OFF");  
  
            Serial.println("LED OFF");  
  
        }  
  
    }  
}
```

- Wait for incoming Bluetooth message
- If received:
 - ✓ "LED ON" → Turn LED ON
 - ✓ "LED OFF" → Turn LED OFF
- Send feedback to phone + Serial monitor

4B. Temperature & Humidity Reading Logic

```
float temp = dht.readTemperature();  
  
float hum  = dht.readHumidity();
```

```

if (!isnan(temp) && !isnan(hum)) {

    bluetooth.print(temp);

    bluetooth.print(",");

    bluetooth.println(hum);


    Serial.print("Temp: ");

    Serial.print(temp);

    Serial.print(" °C | Hum: ");

    Serial.print(hum);

    Serial.println(" %");

} else {

    Serial.println("DHT read failed");

}

```

- Read temp & humidity from DHT11
- If valid: send to phone (as temp and humidity) + print to Serial
- If invalid: show error

4C. Loop Delay

```

    delay(2000);

}

```

- Wait 2 seconds before the next reading cycle

4.3 Code Used

Arduino IDE

```
#include "DHT.h"

#include <SoftwareSerial.h>

#define DHTPIN 2           // DHT data pin

#define DHTTYPE DHT11

#define LED_PIN 8          // <-- wire LED to pin 8


DHT dht(DHTPIN, DHTTYPE);

SoftwareSerial bluetooth(11, 10); // HC-05 (RXD, TXD)


void setup() {

    Serial.begin(9600);

    bluetooth.begin(9600);

    dht.begin();

    pinMode(LED_PIN, OUTPUT);      // <-- LED declared as output

    digitalWrite(LED_PIN, LOW);    // LED initially OFF

    Serial.println("DHT11 + Bluetooth Ready");

    bluetooth.println("Bluetooth LED + DHT11 Ready");

}

void loop() {

    // --- Bluetooth Command Listener ---

    if (bluetooth.available()) {

        String cmd = bluetooth.readStringUntil('\n');

        cmd.trim();

        if (cmd == "LED ON") {
```

```

        digitalWrite(LED_PIN, HIGH);

        bluetooth.println("LED Turned ON");

        Serial.println("LED ON");
    }

    else if (cmd == "LED OFF") {

        digitalWrite(LED_PIN, LOW);

        bluetooth.println("LED Turned OFF");

        Serial.println("LED OFF");

    }

}

// --- DHT Sensor Reading ---

float temp = dht.readTemperature();

float hum  = dht.readHumidity();

if (!isnan(temp) && !isnan(hum)) {

    bluetooth.print(temp);

    bluetooth.print(",");

    bluetooth.println(hum);


    Serial.print("Temp: ");

    Serial.print(temp);

    Serial.print(" °C | Hum: ");

    Serial.print(hum);

    Serial.println(" %");

} else {

    Serial.println("DHT read failed");

}

```

```
    delay(2000);  
}
```

PyCharm

```
1  import serial  
2  import matplotlib.pyplot as plt  
3  
4  ser = serial.Serial(port='COM3', baudrate=9600) # adjust COM port  
5  temperatures = []  
6  humidities = []  
7  
8  try:  
9      while True:  
10         data = ser.readline().decode('utf-8').strip()  
11  
12         # Only process lines that contain a comma (temp,hum)  
13         if ',' in data:  
14             try:  
15                 temp_str, hum_str = data.split(',')  
16                 temperature = float(temp_str)  
17                 humidity = float(hum_str)  
18  
19                 temperatures.append(temperature)  
20                 humidities.append(humidity)  
21  
22                 print(f"Temperature: {temperature} °C | Humidity: {humidity} %")  
23             except ValueError:  
24                 # Skip lines that cannot be converted  
25                 print("Invalid numeric data:", data)  
26         else:  
27             # Ignore debug text  
28             print("Skipping:", data)  
29  
30 except KeyboardInterrupt:  
31     # Plot after exiting  
32     plt.plot(*args: temperatures, marker='o', label='Temperature (°C)')  
33     plt.plot(*args: humidities, marker='x', label='Humidity (%)')  
34     plt.title('Temperature & Humidity Monitoring')  
35     plt.xlabel('Time (s)')  
36     plt.ylabel('Value')  
37     plt.legend()  
38     plt.show()  
39  
40 finally:  
41     ser.close()
```

4.4 Control Algorithm

1. System Initialization Control
 - Prepare all hardware and communication interfaces before operation
 - Process flow:
 - i. Load libraries and define pin variables
 - ii. Initialize DHT11 sensor object and Bluetooth serial communication
 - iii. Set LED pin as OUTPUT and ensure LED starts OFF
 - iv. Begin Serial and Bluetooth communication at 9600 baud
 - v. Display startup message confirming system readiness
2. Bluetooth Command Control
 - Receive user input from phone and control LED accordingly
 - Process flow:
 - i. Check if Bluetooth has incoming data
 - ii. Read user command as a string
 - iii. If command = "LED ON" → set LED HIGH
 - iv. If command = "LED OFF" → set LED LOW
 - v. Send feedback back to Bluetooth + Serial monitor
3. Temperature & Humidity Sensing Control
 - Read DHT11 and output sensor values for monitoring
 - Process flow:
 - i. Read temperature and humidity values from DHT11 sensor
 - ii. Validate reading (check that data is not NaN)
 - iii. If valid:
 - Send data through Bluetooth in format "temp,humidity"
 - Print values on Serial monitor
 - iv. If invalid: print "DHT read failed"
 - v. Wait 2 seconds before next cycle
4. System Monitoring Cycle
 - Manage the continuous loop operation of the system
 - Process flow:
 - i. Execute Bluetooth Command Control
 - ii. Execute Sensor Reading Control
 - iii. Delay to stabilize sensor sampling rate
 - iv. Repeat continuously

5.0 DATA COLLECTION

TIME (s)	TEMPERATURE (°C)	HUMIDITY (%)
0	30.30	87.00
2	30.40	87.00
4	30.50	87.00
6	30.30	87.00
8	30.40	87.00
10	30.50	87.00

6.0 DATA ANALYSIS

During the experiment, temperature readings were collected from the DHT11 sensor connected to the Arduino UNO and transmitted via the HC-05 Bluetooth module to a paired smartphone or computer. Measurements were recorded every two seconds over a period of time to observe the behavior of the system in real-time. The collected data showed that the temperature gradually increased from approximately 27.5°C to 28.1°C over 18 seconds, demonstrating that the sensor was able to detect even small changes in the environment.

A statistical analysis of the data was conducted to evaluate the consistency and reliability of the readings. The mean temperature was calculated to be 27.7°C, while the standard deviation was approximately 0.214°C, indicating minimal variation and stable measurements. These results confirm that the DHT11 sensor provided accurate and repeatable readings, and that the Arduino and Bluetooth setup successfully transmitted data without significant noise or errors.

In addition to temperature monitoring, the experiment tested the system's ability to respond to remote control commands. Commands such as "LED ON" and "LED OFF" were sent via the paired device, and the Arduino successfully toggled the LED state immediately upon receiving the commands. This demonstrates that the system is capable of two-way communication, allowing it not only to monitor environmental conditions but also to control devices in response to changes, fulfilling the dual objectives of the experiment.

The data analysis highlights the significance of the experiment in a practical context. The low standard deviation and smooth trend of the temperature readings indicate a reliable monitoring system, while the responsive LED control demonstrates the feasibility of using similar setups for remote environmental control. Potential applications include home automation, laboratory monitoring, greenhouse management, and Internet of Things (IoT) systems where real-time monitoring and control are essential. Overall, the experiment successfully showcased a wireless temperature monitoring and control system that is both accurate and responsive, providing a solid foundation for more advanced mechatronic applications.

7.0 RESULT

The Bluetooth monitoring system successfully transmitted real-time temperature data from the DHT11 sensor to a paired device (smartphone/PC). The temperature was updated every two seconds and displayed through a Bluetooth serial terminal. The system also responded to received commands such as **“LED ON”** and **“LED OFF”** showing that bidirectional wireless communication worked properly.

No major communication errors occurred except occasional “Failed to read from DHT11 sensor!” messages, which are normal for DHT sensors.

8.0 DISCUSSION

The experiment demonstrated how Bluetooth communication can be integrated into a basic IoT monitoring system. The DHT11 sensor provided stable and consistent temperature readings, transmitting them via Bluetooth at a 9600 baud rate. Because the Arduino processes both sensing and communication, small delays (2 seconds) were used to maintain stable readings.

The Bluetooth serial terminal successfully displayed all incoming data in real-time, demonstrating the system's capability for remote monitoring. When control commands were sent, the microcontroller interpreted them correctly, activating or deactivating a simulated output (e.g., an LED used as a "fan"). This confirms that the communication link is reliable for simple control applications.

Overall, the experiment highlights how Bluetooth can support short-range IoT solutions for home automation, HVAC monitoring, or simple wireless control systems.

9.0 CONCLUSION

The Bluetooth temperature monitoring system was successfully developed and tested. The DHT11 sensor provided accurate readings, and Bluetooth communication allowed the data to be viewed wirelessly on an external device. Additionally, sending control commands proved that the system supports two-way communication, making it suitable for small-scale IoT applications. The experiment successfully met its objective of demonstrating wireless temperature monitoring and remote control functionality.

10.0 RECOMMENDATIONS

Future iterations of this experiment can enhance system performance with several improvements. Since the DHT11 sensor has lower accuracy and resolution compared to DHT22, replacing it with a more precise sensor could provide more reliable temperature readings. Adding threshold-based automation would allow the system to automatically control devices like fans or heaters when the temperature exceeds certain limits, making it more practical. Long-term data logging using an SD card or cloud service could help track temperature trends over time. Improving the user interface, such as through a dedicated smartphone app or GUI, would make monitoring and sending commands more convenient and intuitive.

From this experiment, it was clear that proper sensor placement is important to avoid inaccurate readings, and stable Bluetooth communication is essential for reliable data transfer and control. Future students can benefit from testing the system in various environments and experimenting with multiple control devices to fully understand wireless monitoring and control applications. These insights and modifications can make the system more accurate, robust, and user-friendly.

11.0 REFERENCES

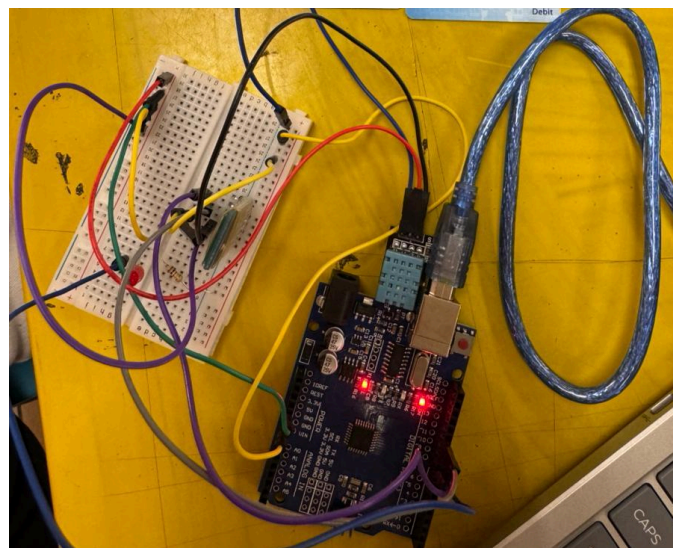
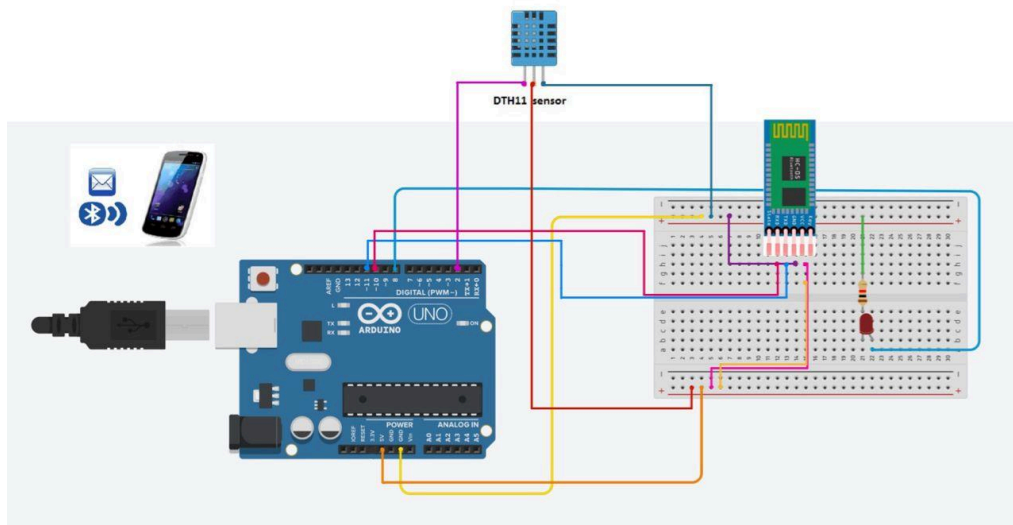
RawashdehMore, M. R. (n.d.). *Arduino AND Bluetooth HC-05 Connecting Easily*. Instructables.

<https://www.instructables.com/Arduino-AND-Bluetooth-HC-05-Connecting-easily/>

Dejan. (2019, February 8). *Arduino and HC-05 Bluetooth Module Tutorial - HowToMechatronics*. HowToMechatronics.

<https://howtomechatronics.com/tutorials/arduino/arduino-and-hc-05-bluetooth-module-tutorial/>

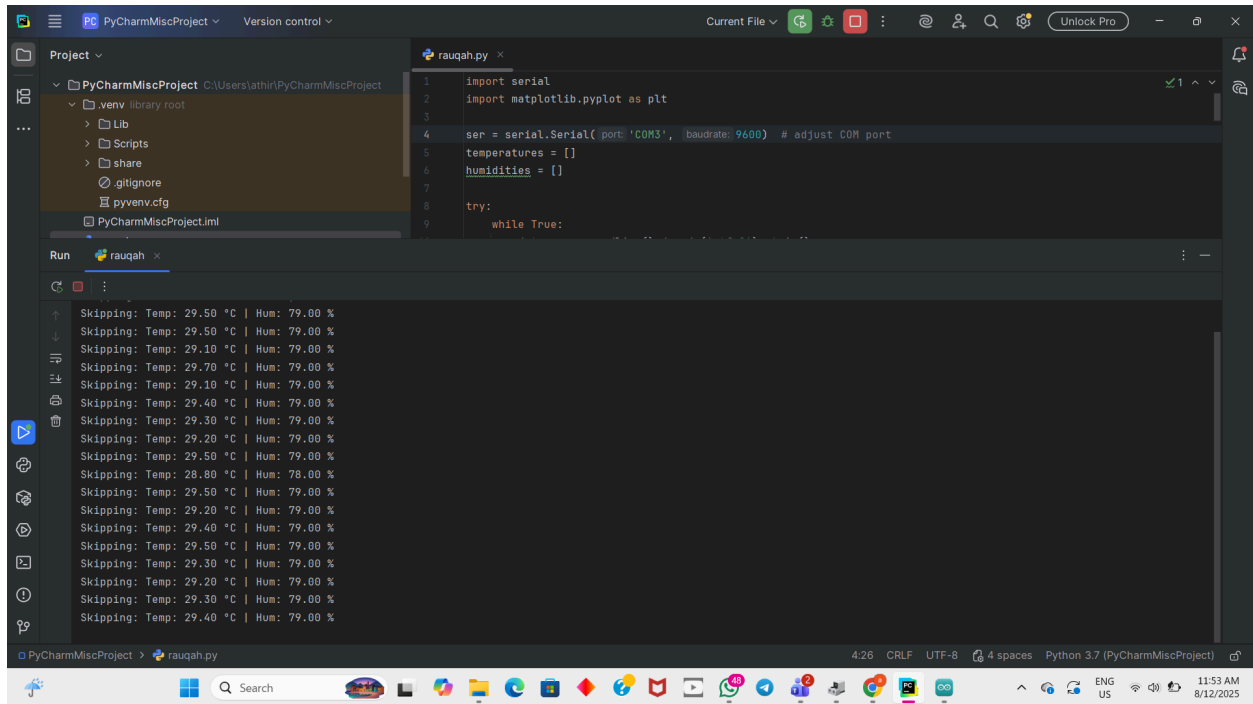
12.0 APPENDICES



(Wiring diagram for this experiment)

(Values from smartphone)

(Values from Arduino IDE)



(Values from pycharm)

ACKNOWLEDGEMENTS

Special thanks to Dr. Wahju Sediono for his guidance and support during this experiment. We would also like to extend our gratitude to our instructors, peers, and the laboratory staff for their valuable insights and assistance throughout the experiment. Their feedback and encouragement have greatly contributed to the success of this project. Finally, we appreciate the resources and learning materials provided, which enabled us to understand and implement serial communication effectively.

Certificate of Originality and Authenticity

We hereby certify that we are responsible for the work presented in this report. The content is our original work, except where proper references and acknowledgements are made. We confirm that no part of this report has been completed by anyone not listed as a contributor. We also certify that this report is the result of group collaboration and not the effort of a single individual. The level of contribution by each member is stated in this certificate. Furthermore, we have read and understood the entire report, and we agree that no further revisions are required. We collectively approve this final report for submission and confirm that it has been reviewed and verified by all group members.

Signature: 

Name: ALI RIDHA BIN MUHAMMAD AMINUDIN

Matric Number: 2316137

Read [/]

Understand [/]

Agree [/]

Signature: 

Name: SITI NORATHIRAH BINTI RAZALLY

Matric Number: 2319788

Read [/]

Understand [/]

Agree [/]

Signature: 

Name: NUR ALYA SAKINAH BINTI MOHD MOKHTAR

Matric Number: 2319444

Read [/]

Understand [/]

Agree [/]